

LEVEL 22 — NumPy File Handling

Goal: Learn how to save, load, read, and write NumPy arrays and CSV files.

1. Saving Arrays with np.save()

`np.save()` stores one NumPy array in a `.npy` file.

Code:

```
import numpy as np

arr = np.array([10, 20, 30, 40])

np.save("data.npy", arr)

print("Array saved successfully")
```

Output:

```
Array saved successfully
```

2. Loading Arrays with np.load()

`np.load()` reads a `.npy` file back into a NumPy array.

Code:

```
import numpy as np

arr = np.load("data.npy")

print(arr)
```

Output:

```
[10 20 30 40]
```

3. Saving Multiple Arrays with np.savez()

`np.savez()` stores multiple arrays in one `.npz` file.

Code:

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

np.savez("data.npz", numbers=a, values=b)

print("Multiple arrays saved")
```

Output:

```
Multiple arrays saved
```

4. Loading Arrays from np.load()

Use the names given when saving to access each array.

Code:

```
data = np.load("data.npz")

print(data["numbers"])
print(data["values"])
```

Output:

```
[1 2 3]
[10 20 30]
```

5. np.savez_compressed()

np.savez_compressed() stores multiple arrays in a compressed .npz file, usually reducing file size.

Code:

```
import numpy as np

arr = np.array([10, 20, 30, 40])

np.savez_compressed("compressed_data.npz", values=arr)

print("Compressed file saved")
```

Output:

```
Compressed file saved
```

6. Writing Text/CSV with np.savetxt()

delimiter=',' means values are separated by commas. fmt='%d' writes integers.

Code:

```
import numpy as np

arr = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

np.savetxt("data.csv", arr, delimiter=",", fmt="%d")

print("CSV written")
```

Output:

```
CSV written

data.csv contains:
10,20,30
40,50,60
```

7. Reading CSV with np.loadtxt()

np.loadtxt() reads numeric text/CSV data. By default, numeric values may be loaded as float.

Code:

```
import numpy as np

arr = np.loadtxt("data.csv", delimiter=",")

print(arr)
```

Output:

```
[[10. 20. 30.]
 [40. 50. 60.]]
```

8. Reading CSV with a Header

skiprows=1 skips the header. usecols selects only Age and Marks. Text columns such as Name cannot be read by plain loadtxt.

Code:

```
import numpy as np

# File:
# Name, Age, Marks
# Amit, 20, 85
# Ravi, 21, 90

data = np.loadtxt(
    "students.csv",
    delimiter=",",
    skiprows=1,
    usecols=(1, 2)
)

print(data)
```

Output:

```
[[20. 85.]
 [21. 90.]]
```

9. np.genfromtxt() for Missing Values

genfromtxt() is useful when data contains missing values. np.nan represents a missing numeric value.

Code:

```
import numpy as np

# File:
# 10, 20, 30
# 40, , 60
# 70, 80, 90

data = np.genfromtxt(
    "missing.csv",
    delimiter=",",
    filling_values=np.nan
)

print(data)
```

Output:

```
[[10. 20. 30.]
 [40. nan 60.]
 [70. 80. 90.]]
```

10. Writing CSV with Header

header adds column names. comments=" prevents NumPy from putting # before the header.

Code:

```
import numpy as np

data = np.array([
    [20, 85],
    [21, 90]
])

header = "Age, Marks"

np.savetxt(
    "students.csv",
    data,
    delimiter=",",
    header=header,
```

```

        comments="",
        fmt="%d"
    )

    print("CSV with header written")

```

Output:

```

CSV with header written

students.csv contains:
Age,Marks
20,85
21,90

```

11. Reading CSV with `genfromtxt()` and Header

`names=True` uses the first row as column names.

Code:

```

import numpy as np

data = np.genfromtxt(
    "students.csv",
    delimiter=",",
    names=True
)

print(data.dtype.names)
print(data["Age"])
print(data["Marks"])

```

Output:

```

('Age', 'Marks')
[20. 21.]
[85. 90.]

```

12. Delimiters

A delimiter tells NumPy how columns are separated. Common delimiters are comma ',', semicolon ';', tab '\t', and space ' '.

Code:

```

import numpy as np

# File uses semicolon:
# 10;20;30
# 40;50;60

data = np.loadtxt("data.txt", delimiter=";")

print(data)

```

Output:

```

[[10. 20. 30.]
 [40. 50. 60.]]

```

13. CSV with Missing Values and Custom Missing Marker

`missing_values` identifies the missing marker. `filling_values` specifies what NumPy should put in its place.

Code:

```

import numpy as np

# File:

```

```

# 10,20,30
# 40,NA,60

data = np.genfromtxt(
    "data.csv",
    delimiter=",",
    missing_values="NA",
    filling_values=np.nan
)

print(data)

```

Output:

```

[[10. 20. 30.]
 [40. nan 60.]]

```

14. Complete Mini Example

This combines saving/loading NumPy format and CSV format.

Code:

```

import numpy as np

# Create data
marks = np.array([
    [80, 90, 85],
    [70, 75, 80]
])

# Save as NumPy file
np.save("marks.npy", marks)

# Load NumPy file
loaded = np.load("marks.npy")

# Save as CSV
np.savetxt("marks.csv", marks, delimiter=",", fmt="%d")

# Load CSV
csv_data = np.loadtxt("marks.csv", delimiter=",")

print("Original:")
print(marks)

print("Loaded from .npy:")
print(loaded)

print("Loaded from CSV:")
print(csv_data)

```

Output:

```

Original:
[[80 90 85]
 [70 75 80]]

Loaded from .npy:
[[80 90 85]
 [70 75 80]]

Loaded from CSV:
[[80. 90. 85.]
 [70. 75. 80.]]

```

Quick Revision — LEVEL 22

np.save() → save one NumPy array as .npy

np.load() → load .npy or .npz data

np.savez() → save multiple arrays in .npz

np.savez_compressed() → compressed .npz

np.savetxt() → write text/CSV

np.loadtxt() → read numeric text/CSV

np.genfromtxt() → read text/CSV with missing values and more flexible handling

delimiter → tells NumPy how columns are separated

header → column names at the top of a CSV/text file

skiprows → skip rows such as headers

names=True → use the first row as field names with genfromtxt()

np.nan → commonly used to represent a missing numeric value